

Second post: two of TT's own systems, shipped bare, so you can run them.

A public follow-up to the first post · 5 October 2026 · written by two AI assistants from the originator's notes and direction · the first post, in English and Norwegian, is at the same address

What this post is. The first post said what TT is: a developing way for people and AI systems to investigate, make, correct and share work together, under three tenets. It also said what TT was not yet: proof. This post is the first piece of that proof, in the only form proof should take on a public page: things you can run. Two of the systems TT has used on itself every day since September are now in the repository as templates, each bare, each with a checker and a self-test, each with an optional layer of worked examples kept apart from the bare files.

Why bare. A template that arrives full of someone else's vocabulary is a template you will spend a week emptying. Each of these ships with just enough entries to show every rule once: one confirmed term, one unresolved term, one pair of look-alikes kept apart; one issue, one observed error, one fix trial. Everything else is yours to add. The optional layer, where there is one, is a screened extract of TT's own records, so you can see the template carrying real weight, and delete it the moment you have your own.

Why a checker. A rule that has to be remembered has already failed. That line is TT's, learned the hard way, and it is why each template comes with a small plain-Python script, nothing to install, that refuses the mistakes the rule exists to prevent. The checker does not know whether your meanings are true or your fixes are good. It knows whether the record is well formed, whether something was promoted without its owner's word, whether two things that look alike have been told apart in writing. That is a smaller promise than it sounds, and it is the one a machine can keep.

Where it lives. The repository is public under the same licences as the first post: AGPL-3.0 for code, CC BY-SA 4.0 for writing. The [templates/](#) folder holds both systems; the README points at each. Take anything, give credit, give back the same way.

The dictionary: one vocabulary, three views, one rule.

The problem it solves. Every project grows its own words. Some are coined on purpose; most arrive sideways, used before anyone has said what they mean, and by the time two people disagree about one, each has been using it for a month. Worse: words that resonate get promoted. A phrase that sounds right is treated as settled because it sounds right, and the person who coined it is never asked. TT calls this silent promotion, and the dictionary exists to make it impossible.

The one rule. A term is **Confirmed** only when its owner has stated its meaning plainly, in their own words, and that statement is written on the entry. Until then it is **Unresolved**: a reading someone made, marked as a reading, kept so it can be checked, never promoted by habit. The checker refuses a Confirmed term that carries no owner statement.

Three views. The same vocabulary is held from three sides. [HUMAN.md](#) holds the words people use, as they use them. [FORMAL.md](#) holds the words a tool or a rule gives a fixed meaning to, and each of those entries must say where that meaning executes, so a word can be read against the thing that enforces it. [MATH.md](#) holds the words with a number or a test behind them, and each must say how it is measured. All three share one line grammar, so one checker reads them all, and a line that looks like an entry and does not parse is an error, not a warning.

Look-alikes. Two terms can share an outward shape, the same act, the same first letters, the same sound, and mean different things underneath. The dictionary calls these a look-alike pair and keeps them apart with a crosswalk on each side that names the other and says how they differ. The template's own example is handoff and dropoff: both pass something over; in one the giver is still there to answer, in the other nobody is present. Same signature, different thing. The checker flags a pair whose names share a stem and carry no crosswalk, so the difference is written down before someone assumes it away.

What you run. `python3 check_dictionary.py` checks the three files beside it and prints one line: how many entries, how many confirmed, and a digest. Two people who hold the same dictionary get the same digest; that is how you confirm, in one line, that you are reading from the same page. `--selftest` runs the checker on itself. `EXPECTED-OUTPUT.txt` sits beside it so you can compare.

The optional layer. `optional/TT-COLLECTED.md` is fourteen of TT's own terms in the template's grammar, each with the public wording its author approved. Three of them repeat the bare template's examples on purpose, so you can hold the example and the real entry side by side.

Issue and error tracking: separate records, a score in the open, a checker.

The problem it solves. Most issue lists mix four things into one line: the problem, the times it was seen, the fixes that were tried, and the edits someone made to the record. Once they are mixed, nobody can say how many times a thing actually happened, which fix was actually tested, or whether a closed issue was closed by evidence or by tidiness. The template keeps the four apart: an issue, an observed error, a fix trial and a record change are each their own record, linked by id.

The score. Each issue can carry five dimensions, scored one to five: impact, reach, recurrence, persistence, and how hard the change is to roll back. The weighted total runs from ten to fifty and is a triage number, nothing more. Confidence and the cost of verifying stay separate from it, on purpose: a loud issue you are unsure of and a quiet one you have proven are different animals, and a single number that averaged them would hide exactly what you need to know. A missing dimension leaves the total unscored rather than defaulted; a synthetic pass does not count as a repair in use.

What you run. `python3 track.py init ledger.jsonl` makes an empty ledger. `check` validates it: structure, references between records, scores, declared test scope, recorded screening. `report` builds a local report, and `report --public` a lighter one that omits private titles, owners, steps and origin pointers, so a problem whose

supporting record must stay private can still be described in public. `python3 -m unittest test_track.py` runs twenty-nine tests. The optional layer is a fictional ledger, nothing from TT's own records, there to show the forms filled in.

How to report to TT. The repository now carries two forms beside the usual bug report: an issue report, for a problem that needs investigation or a change, and an error observation, for one event seen once, without assuming its cause. Both are plain text; both ask for what you expected, what you observed, and reproduction steps if you have them, or the honest line that reproduction is unknown. For a wording or translation concern, a short excerpt and your proposed wording is enough. Causes stay labelled unknown, suspected or demonstrated until the evidence moves them. The full policy, in English and Norwegian, is in the template folder.

What comes next. The first post promised the method; this one hands over two of its tools. A third, the research lab TT runs its stranger questions through, is held back for a later post, because it is still the part of TT that is not always reliable, and a tool goes public here only when its checker can say so.

Licence: the writing on this page is CC BY-SA 4.0, the code in the templates AGPL-3.0. Written with AI help, from the originator's notes and direction. The Norwegian edition of this post is at the same address.